

Sistema RAG para Valorización Automática de Licitaciones Mineras

Recuperación Semántica, Estimación Gaussiana con Ajuste IPC
y Gradient Boosting Jerárquico

Daniel Floridor

Construcción Civil

INF398 - Introducción al Machine Learning

Julio 2026

Resumen

Se presenta un sistema de valorización automática de licitaciones de obras mineras en Chile, construido sobre un pipeline de recuperación semántica aumentada con generación (RAG). El sistema combina SBERT+FAISS para recuperar partidas históricamente similares, un modelo Gaussiano ponderado para estimar el precio unitario (P.U.) con ajuste al Índice de Precios al Consumidor (IPC), y un ensemble XGBoost jerárquico—con un modelo por grupo semántico-unidad—como mecanismo de respaldo cuando no existen referencias directas. Los datos provienen de 11 archivos ECO (Estimaciones de Costos de Operación) de licitaciones adjudicadas en MercadoPublico.cl (2022–2026), totalizando 3 804 registros distribuidos en 45 grupos. El análisis revela un comportamiento bimodal: los grupos con semántica homogénea alcanzan $R^2 \geq 0,83$ y $MAPE < 15\%$, mientras los grupos `global_otros` y `unidad` presentan $MAPE > 200\%$ por heterogeneidad semántica residual que el clustering K-Means no resuelve. Se identifican las limitaciones del agrupamiento automático para ítems de suma alzada y se propone una taxonomía keyword-first como extensión.

1. Introducción y descripción del problema

La estimación de costos en licitaciones de obras civiles y mineras es una tarea de alto impacto económico. En Chile, las licitaciones públicas cualquier empresa puede adjuntar un propuesta, y las empresas mandantes deben establecer un presupuesto referencial *antes* de recibir ofertas de contratistas, puede ser este publico o para controlar algunos items para realizar futuras negociaciones. Este presupuesto referencial se estima a partir de antecedentes históricos de proyectos similares, expresados en Estimaciones de Costos de Operación (ECO), en el caso de la minería—itemizados en formato Excel que detallan partidas de trabajo con su unidad de medida, cantidad y precio unitario.

1.1. Características del problema

El problema presenta tres dificultades técnicas centrales documentadas en la literatura:

1. **Heteroscedasticidad severa.** Un mismo itemizado contiene partidas cuyo P.U. varía desde \$800 CLP/kg (acero) hasta \$2.000.000 CLP (movilización de equipos). Un modelo de regresión único falla porque los primeros splits del árbol se destinan a separar las sumas alzadas (gl) del resto, dejando sin potencia predictiva a las partidas de mayor frecuencia. Alshboul. O et al. (2022) documentan que XGBoost supera a ANN y RF en estimación de costos de construcción *solo cuando los datos son categóricamente homogéneos*.
2. **Deriva temporal (IPC).** Los registros son históricos existen ECOs del 2022 otro del 2025 y otros del 2026, (2022-2026) período con inflación acumulada $\approx 28\%$ según el Banco Central de Chile. Comparar precios nominales de 2022 con los de 2026 introduce sesgo sistemático equivalente a dicho porcentaje.
3. **Ambigüedad léxica.** Descripciones semánticamente idénticas aparecen con terminología distinta en cada licitación (*Excavación en Zanja Suelo Común* vs. *Excavación Total*). La recuperación basada en TF-IDF/BM25 trata estos como documentos distintos, fallando en el recall (Lewis. P et al. 2020).

1.2. Datos

El dataset de entrenamiento se construyó a partir de 11 archivos ECO en formato `.xlsx`:

- 3 archivo real (ECO adjudicad, app 137 - 300 partidas cada uno).
- 8 archivos sintéticos generados mediante variación estocástica $\pm 20\%$ sobre los precios base del ECO real, con factores IPC proporcionales a la fecha de cada documento (2022 a 2026).

El script `build_historical.py` extrae las partidas y genera `data/historical.csv` con 3 804 registros y las columnas: `descripcion`, `unidad`, `cantidad`, `año`, `mes`, `ipc_factor`, `precio_unitario`. La columna `ipc_factor` se calcula como:

$$\text{ipc_factor} = \frac{\text{IPC}(a_{\text{hist}}, m_{\text{hist}})}{\text{IPC}(a_{\text{actual}}, m_{\text{actual}})} \quad (1)$$

donde a y m denotan año y mes, respectivamente, y los valores provienen del archivo `IPC_VAR_MEN1_HIST_NEW.csv`, data extraída del Banco Central de Chile.

2. Solución propuesta

El sistema implementa un pipeline RAG de tres etapas con lógica de orquestación por prioridad (Se implemento un boton para solo aplicar XGboost para evaluar su rendimiento):

$$\hat{p}(q) = \begin{cases} \mu_{\text{Gauss}}^*(q) & \text{si } \exists \text{ exact match (score} = 1,0) \\ \mu_{\text{Gauss}}^*(q) & \text{si } \exists \text{ refs con score } \geq 0,70 \\ \hat{y}_{\text{XGBoost}}(q) & \text{en otro caso (fallback)} \end{cases} \quad (2)$$

2.1. Etapa 1: Recuperación semántica (SBERT + FAISS)

Se emplea el modelo `paraphrase-multilingual-mpnet-base-v2` de SBERT (Reimers. N y Gurevych. I, 2019) —un encoder Transformer de 12 capas con *mean pooling* sobre embeddings de tokens—para proyectar tanto las partidas del índice histórico como el query de la consulta a $\mathbb{R}^{768}$. Los vectores se normalizan con norma L2, de modo que la similitud coseno equivale al producto punto, y se almacenan en un índice FAISS `IndexFlatIP` (J. Johnson, et al. 2021) que permite búsqueda aproximada de vecinos más cercanos en $\mathcal{O}(\log n)$.

La superioridad del dense retrieval sobre TF-IDF para consultas con variación léxica está documentada por Lewis. P et al. (2020).

2.2. Etapa 2: Estimación Gaussiana con ajuste IPC

El modelo Gaussiano pondera los precios ajustados de las k partidas recuperadas por sus scores de similitud:

$$\mu^* = \frac{\sum_{i=1}^k w_i \cdot p_i^{\text{ajust}}}{\sum_{i=1}^k w_i}, \quad \sigma^* = \sqrt{\frac{\sum_{i=1}^k w_i (p_i^{\text{ajust}} - \mu^*)^2}{\sum_{i=1}^k w_i}} \quad (3)$$

donde w_i es el score FAISS y $p_i^{\text{ajust}} = p_i^{\text{nom}} \times (\text{IPC}_{\text{actual}} / \text{IPC}_{\text{hist}_i})$.

2.3. Etapa 3: Ensemble XGBoost jerárquico (fallback)

Cuando el modelo Gaussiano no dispone de referencias con $\text{score} \geq 0,70$, se activa el ensemble XGBoost. La solución clave, fundamentada por la heteroscedasticidad descrita en la Sección 1, es entrenar un **modelo independiente por grupo semántico-unidad** (K-means) (Chen. T y Guestrin. C, 2016).

2.3.1. Agrupación jerárquica en dos niveles

1. **Nivel 1 — grupo_unidad**: mapeo determinista de la unidad de medida (hh, hm, m3, kg, ml, gl, un, m2, otros).
2. **Nivel 2 — sub-cluster**: dentro de cada grupo_unidad, K-Means (Chen. T y Guestrin. C, 2016) con $k = 4$ sobre embeddings SBERT de las descripciones. Para grupos con keywords dominantes (excavacion_roca, hormigon, tuberia, movilizacion, etc.), una taxonomía regex tiene prioridad sobre K-Means.

La clave compuesta resultante (volumetrico_excavacion_roca, mano_obra_Supervisor, etc.) define el subconjunto de entrenamiento de cada sub-modelo. Los grupos con $n < 15$ registros usan la mediana como estimador simple.

2.3.2. Features del modelo

Cada sub-modelo XGBoost recibe exclusivamente cuatro variables numéricas y una binaria:

$$\mathbf{x} = [\log_1(1 + \text{cantidad}), \text{año}, \text{mes}, \text{ipc_factor}, \text{Adjudicado o no}]^T \quad (4)$$

El número de ítem, el nombre del contratista y el código de licitación se excluyen deliberadamente para evitar sobreajuste a identificadores sin valor predictivo, esto genero sesgos al inicio, debido a que si se ingresaba una misma empresa, esta tomaba con mayor asertividad sus precios pasados, por este motivo se excluyeron, pero si estan como contexto de orden.

2.3.3. Entrenamiento y Historial

El target es $\log_1(1 + \text{precio_unitario})$ para estabilizar el entrenamiento en rangos de precios amplios. Las métricas se calculan sobre predicciones devueltas a escala original mediante $\exp(y) - 1$. El modelo completo (45 sub-modelos, encoders y K-Means) se serializa con `joblib` junto al MD5 del CSV de entrenamiento, evitando reentrenamiento en reinicios si los datos no cambiaron.

3. Implementación

3.1. Arquitectura de módulos

Cuadro 1: Módulos principales del sistema

Módulo	Responsabilidad	Dependencias clave
nlp/rag_index.py	Indexación FAISS + parsing ECO xlsx	FAISS, openpyxl, SBERT
nlp/valuation_service.py	Orquestación Gausiano/XGBoost	rag_index, estimadores
nlp/gaussian_estimator.py	Estimación $\mu^* \pm \sigma^*$ + IPC *es mas similar a un modelo gaussiano de una capa	numpy, sklearn
nlp/xgboost_estimator.py	Ensemble jerárquico, aplicación de random forest con XGboost	xgboost, sklearn, joblib
nlp/macro_utils.py	Carga IPC, extracción fecha EETT	pandas, re
build_historical.py	ETL: ECO xlsx $\rightarrow$ historical.csv * Solo aplica cuando no existe un csv	openpyxl, pandas
app.py	Flask API + rutas /valorizar	Flask, sentence-transformers

3.2. Decisiones de diseño relevantes

Log-scale en el target XGBoost. Entrenar sobre $\log_1(1 + p)$ comprime el rango [\$800, \$2,000,000] a [6.7, 21.4] en escala natural, evitando que los primeros splits se destinen a separar gl-items del resto. Sin esta transformación se observó $\text{MAPE} = 138\%$, $R^2 = -0,30$ en el modelo monolítico.

Selección (Ec. 2). El XGBoost nunca sobrescribe un exact match del modelo Gaussiano, cuyo score = 1,0 implica certeza total ($\sigma^* = 0$). El error previo—donde XGBoost predicó \$133 M para una partida con tres referencias exactas a \$16 568—fue consecuencia de una lógica de sobreescritura incondicional.

Persistencia El archivo `data/xgb_model.pkl` almacena el MD5 del CSV de entrenamiento.

Regularización El modelo se regula segun el tipo de ECO, si la empresa adjudico o no la licitacion, se le asigna un peso de 1 a los que si, y 0.3 a los que no, esto con el fin de darle prioridad a los que si ganaron la licitacion.

3.3. Fragmento ilustrativo

El siguiente fragmento resume la lógica de predicción del ensemble:

Listing 1: Predicción en XGBoostValuationModel.predict()

```

1 grupo_u = GRUPO_UNIDAD.get(unidad.lower(), 'otros')
2 subcat = _asignar_subcategoria(descripcion, grupo_u) # regex
   taxonomy
3 if subcat is None and grupo_u in self.kmeans_por_grupo: # K-Means
   fallback
4     emb = self._embedder.encode([descripcion])
5     subcat = f"otros_{self.kmeans_por_grupo[grupo_u].predict(emb)
   [0]}"
6 clave = f"{grupo_u}_{subcat}"
7 if clave in self.modelos:
8     x = [[log1p(cantidad), ano, mes, ipc_factor]]
9     y_log = self.modelos[clave].predict(x)[0]
10    return expm1(y_log) # escala original

```

4. Metodología experimental y resultados

4.1. Configuración experimental

El entrenamiento usa validación cruzada K-Fold con $k = 5$ para grupos con $n \geq 10$ registros; grupos con $n < 10$ aplican la mediana. Las métricas se calculan sobre las predicciones en escala original, por lo que son honestas y no infladas por overfitting.

Las métricas reportadas son:

- MAE (\$): error absoluto medio.
- MAPE (%): error porcentual absoluto medio, sensible a valores cercanos a cero.
- F1 Score (%): precisión y recall.
- R^2 : coeficiente de determinación; < 0 indica peor que predecir la media.

4.2. Resultados por grupo

La Tabla 2 presenta los resultados de los 38 grupos entrenados con XGBoost (5-fold CV), ordenados por R^2 descendente. Los 7 grupos restantes usan mediana simple por insuficiencia de datos.

Cuadro 2: Resultados XGBoost 5-fold CV por grupo semántico-unidad

Grupo	n	MAE (\$)	MAPE (%)	R^2	F1
global_movilizacion	31	99,936,975	14.1	0.91	0.410
volumetrico_excavacion_roca	43	10,857	14.6	0.89	0.772
otros_otros_1	42	3,339,477	67.4	0.87	0.502
lineal_otros_1	164	44,228	32.8	0.84	0.837
volumetrico_otros_1	160	137,551	46.8	0.83	0.735
area_otros_3	41	602	10.5	0.83	0.490
otros_otros_3	74	9,608,274	2930.3	0.76	0.669

continúa en la próxima página

Cuadro 2 (continuación)

Grupo	n	MAE (\$)	MAPE (%)	R ²	F1
otros_otros_0	21	3,650,716	9.4	0.69	0.565
lineal_otros_0	50	147,808	41.3	0.60	0.447
area_otros_1	133	4,493	30.9	0.59	0.659
peso_otros_2	41	5,897	72.3	0.58	0.650
lineal_otros_2	70	288,683	47.1	0.58	0.582
lineal_otros_3	181	688,551	229.7	0.55	0.572
global_instalacion_faena	21	7,733,911	7.6	0.49	0.433
volumetrico_otros_3	115	126,287	112.4	0.49	0.707
lineal_tuberia	241	227,533	161.5	0.47	0.620
volumetrico_hormigon	303	257,428	39.4	0.46	0.609
volumetrico_relleno	577	13,367	46.4	0.36	0.640
unidad_otros_2	391	3,001,623	153.4	0.29	0.560
area_otros_2	81	29,019	27.3	0.29	0.548
mano_obra_tecnico	40	1,596	8.5	0.25	0.303
volumetrico_otros_0	343	85,209	131.8	0.21	0.510
area_otros_0	30	60	22.8	0.21	0.324
peso_otros_0	40	11,373	20.6	0.20	0.412
peso_acero	160	9,103	85.8	0.14	0.419
unidad_otros_3	321	3,323,295	86.3	0.10	0.512
unidad_otros_0	453	9,141,349	213.8	0.10	0.686
volumetrico_otros_2	190	33,849	78.4	0.10	0.518
volumetrico_material_dren	154	5,125	24.1	0.01	0.464
otros_otros_2	30	32,841	476.5	-0,02	0.356
volumetrico_excavacion_suelo	519	113,877	129.2	-0,02	0.495
mano_obra_operario	30	540	4.1	-0,03	0.167
peso_otros_1	40	1,515	4.6	-0,08	0.181
global_otros_1	91	1,076,333,943	356.4	-0,15	0.394
unidad_otros_1	362	23,004,807	368.6	-0,18	0.221
global_otros_2	31	8,492,577,610	1237.6	-0,37	0.381
global_otros_0	20	693,354,574	185.3	-0,42	0.138
global_otros_3	32	1,480,618,599	1727.4	-1,85	0.174
Promedio ponderado	5,417	77,973,807	174.7	0.258	0.544

4.3. Resumen por categoría de rendimiento

La Tabla 3 categoriza los 38 grupos con XGBoost.

Cuadro 3: Distribución de rendimiento por categoría

Categoría	Grupos	Criterio	Ejemplos
Excelente	6	$R^2 \geq 0,80$, MAPE < 50 %	volumetrico_excavacion_roca, global_movilizacion
Aceptable	10	$R^2 \in [0,4, 0,8)$	volumetrico_hormigon, lineal_otros_1
Mediocre	10	$R^2 \in [0,0, 0,4)$	peso_acero, volumetrico_relleno
Inutilizable	12	$R^2 < 0$ ó MAPE > 200 %	global_otros_*, unidad_otros_*

5. Análisis crítico de los resultados

5.1. Grupos de alto rendimiento

Los grupos `volumetrico_excavacion_roca` ($R^2 = 0,89$, MAPE= 14,6 %) y `global_movilizacion` ($R^2 = 0,91$, MAPE= 14,1 %) demuestran que el acierto es correcto cuando la semántica intra-grupo es homogénea. En ambos casos, las partidas comparten no solo la unidad sino el proceso productivo (roca no ripiable, movilización de equipos), por lo que `cantidad`, `año` e `ipc_factor` son parametros genuinamente informativos, en el caso de que ambos estuvieran en un solo grupo, generaria un resultado muy mediocre.

`area_otros_3` ($R^2 = 0,83$, MAPE= 10,5 %) agrupa vegetación (*Roce*, *tala*, *Destron-que*), cuyo precio por m² depende fuertemente del IPC y del año—relación que XGBoost captura correctamente, son precios más estandar es decir es facil predecible este grupo.

5.2. Anomalía: MAPE alto con R^2 alto

`otros_otros_3` exhibe MAPE= 2930,3 % pero $R^2 = 0,76$. Este aparente contrasentido se explica porque el grupo contiene partidas con rangos de precio de varios órdenes de magnitud (*Agotamiento túnel evacuador* vs. *Tala de árboles*): el modelo captura el *ranking* correcto (R^2 alto) pero falla en el valor absoluto de las partidas más pequeñas, lo que infla el MAPE. Es un caso de fallo del K-Means: ítems semánticamente heterogéneos quedaron en el mismo cluster. Este es un gran detalle del algoritmo, al centrarse en la semántica, a veces omite algo tan claro como los valores altos y dispares.

5.3. Grupos inutilizables: las sumas alzadas (gl)

Los cuatro subgrupos `global_otros_*` presentan R^2 negativo y MAPE > 180 %. El análisis de los ejemplos revela la causa:

- `global_otros_3`: *Gasto a costo efectivo* y *Gastos Generales*. Estos ítems son contratos administrativos cuyo valor se negocia caso a caso y no tiene relación funcional con `cantidad`, `año` ni `ipc_factor`. La variabilidad es *intrínseca al proceso de licitación*, no reducible con ML.

- **global_otros_2: Ducto de Barras** (equipo eléctrico de alta tensión, \$B de CLP) junto a *Asesoría Experta Geotécnica* (honorarios, \$M de CLP). El K-Means los agrupó por similitud superficial de embedding, pero son categorías económicamente incomparables.

La solución no es refinar el modelo sino **excluir estos ítems del scope del XGBoost**, dado que son inherentemente no predecibles a partir de parametros estructurales.

5.4. Caso crítico: volumetrico_excavacion_suelo

Con $n = 519$ (el grupo más grande de volumétrico), $R^2 = -0,02$ indica que XGBoost es peor que predecir la media. Los ejemplos del grupo (*Excavación en Zanja Suelo Común sin Agotamiento* y *Excavación Total*) revelan el problema: *Excavación Total* es una partida suma que agrega múltiples subpartidas de precio heterogéneo, mientras que *Excavación Zanja* tiene precio unitario relativamente estable. El K-Means no pudo separarlos porque sus embeddings SBERT son próximos (*excavación* aparece en ambos), pero sus distribuciones de precio son radicalmente distintas. La corrección es un keyword regex explícito: `excavacion_total` debe ser una clave separada de `excavacion_suelo`.

5.5. Anomalía estadística: MAPE bajo con R^2 negativo

`mano_obra_operario` (MAE= \$540, MAPE= 4,1 %, $R^2 = -0,03$) y `peso_otros_1` (MAPE= 4,6 %, $R^2 = -0,08$) presentan esta combinación contraintuitiva. La explicación es la baja varianza del target: cuando todos los P.U. son prácticamente iguales (p.ej. todos los ayudantes cobran $\approx$ \$22 000/hh), la media ya es un predictor casi perfecto y XGBoost no aporta sobre ella—de ahí $R^2 \approx 0$ —pero el MAPE es bajo porque los errores absolutos son pequeños respecto al valor nominal. En este caso, el modelo Gaussiano ponderado (de una capa o simple) es estrictamente superior.

6. Conclusiones y trabajo futuro

6.1. Conclusiones

1. **La agrupación jerárquica es necesaria pero insuficiente.** K-Means en el espacio SBERT captura similitud semántica superficial pero no discrimina entre ítems económicamente incompatibles que comparten vocabulario (e.g. *Excavación Total* vs. *Excavación Zanja*). Una discriminación por precios es necesaria.
2. **Los ítems global_otros son no predecibles.** Gastos generales, honorarios de consultoría y ítems contractualmente negociados no tienen relación funcional con los parametros disponibles. Incluirlos degrada las métricas globales sin aportar valor predictivo.
3. **La normalización IPC es condición necesaria.** Sin ajuste por inflación, los precios nominales de 2022 subestiman los de 2026 en $\approx 28\%$, introduciendo sesgo sistemático en el Gaussiano y en el XGBoost.
4. **El pipeline Gaussiano + XGBoost fallback es correcto.** Los grupos de alto rendimiento ($R^2 > 0,80$) son aquellos donde el Gaussiano ya encuentra referencias

exactas; el XGBoost actúa solo cuando no las hay, precisamente donde el embedding no ayuda.

5. **La log-transformación del target es imprescindible.** Sin ella, MAPE= 138 %, $R^2 = -0,30$ en el modelo monolítico; con ella y agrupación jerárquica, los mejores grupos alcanzan $R^2 = 0,91$.

6.2. Trabajo futuro

- **Grupos Discriminados.** Se aplica principalmente a los semanticamente similares, pero numericamente distintos.
- **Exclusión de ítems intrínsecamente aleatorios.** Los grupos `global_otros_*` con $R^2 < 0$ deben recibir una bandera de `no_predecible` y derivarse al modelo Gaussiano con margen de incertidumbre alto (σ^* amplificado).
- **TabPFN para grupos pequeños.** Para grupos con $n < 50$ donde K-Fold CV es inestable, evaluar TabPFN como alternativa sin hiperparámetros.
- **Fine-tuning SBERT.** Con pares de sinónimos del dominio minero chileno (*Excavación Total* $\sim$ *Mov. de tierras*) para mejorar la separación de clusters K-Means y reducir los falsos positivos de recuperación semántica.